Gattoc, Mika Mclaren O.

Python Laboratory Activity: Lists and Tuples Advanced Level • 10 Programming Problems • Hands-on Laboratory Exercise

General Instructions

1. Create a Python program for each of the 10 problems below.
2. Use appropriate Python lists, tuples, loops, conditional statements, functions, and list methods.
3. Apply logical problem-solving techniques. Avoid hardcoding results that should be computed by your program.
4. Display clear, organized, and properly labeled output.
5. Test each program using the given sample data and at least one additional test case.
6. Save each program using the filename format `ListTuple_Lab01_LastName.py`, changing the activity number for each problem.

Problem 1: Student Grade Analysis System

Difficult

Create a program that processes student records stored as tuples inside a

list. Sample data:

```python
students = [
    ("Ana", (85, 90, 88)),
    ("Ben", (78, 82, 80)),
    ("Carlo", (92, 95, 91)),
    ("Dina", (88, 84, 90)),
    ("Eric", (70, 75, 72)),
    ("Faith", (95, 93, 97))
]
```

```python
students = [
    ("Ana", (85, 90, 88)),
    ("Ben", (78, 82, 80)),
    ("Carlo", (92, 95, 91)),
    ("Dina", (88, 84, 90)),
    ("Eric", (70, 75, 72)),
    ("Faith", (95, 93, 97))
]

averages = [(name, sum(grades) / len(grades)) for name, grades in students]

print("STUDENT GRADE REPORT")
for name, avg in averages:
    print(f"{name}: Average = {avg:.2f}")

top = max(averages, key=lambda x: x[1])
print(f"\nTop Student: {top[0]}")

qualified = [name for name, avg in averages if avg >= 85]
print(f"Qualified Students: {qualified}")

sorted_students = sorted(averages, key=lambda x: x[1], reverse=True)

print("\nSorted Results:")
print(sorted_students)
```

```
STUDENT GRADE REPORT
Ana: Average = 87.67
Ben: Average = 80.00
Carlo: Average = 92.67
Dina: Average = 87.33
Eric: Average = 72.33
Faith: Average = 95.00

Top Student: Faith
Qualified Students: ['Ana', 'Carlo', 'Dina', 'Faith']

Sorted Results:
[('Faith', 95.0), ('Carlo', 92.66666666666667), ('Ana', 87.66666666666667), ('Dina', 87.33333333333333), ('Ben', 80.0), ('Eric', 72.33333333333333)]
```

Problem 2: Duplicate Removal Without Losing Order Difficult

Write a program that removes duplicate values from a list while preserving the order of their first occurrence.

Sample data:

```python
numbers = [
    12, 5, 8, 12, 3, 5, 9, 8,
    15, 3, 20, 12, 9, 25, 15
]
```

Requirements:

1. Create a new list containing only unique values.
2. Do not use `set()` to remove duplicates.
3. Count how many times each number appears.
4. Identify the most frequently occurring number or numbers.
5. Convert the unique list into a tuple.
6. Display the original list, unique list, frequency report, and tuple.

Challenge: If multiple numbers have the same highest frequency, display all of them.

```python
numbers = [
    12, 5, 8, 12, 3, 5, 9, 8,
    15, 3, 20, 12, 9, 25, 15
]

unique_numbers = list(dict.fromkeys(numbers))

counts = {num: numbers.count(num) for num in unique_numbers}

max_count = max(counts.values())
most_frequent = [num for num, count in counts.items() if count == max_count]

unique_tuple = tuple(unique_numbers)

print("Original List:", numbers)
print("Unique List:", unique_numbers)

print("\nFrequency Report:")
for num, count in counts.items():
    print(f"Number {num}: appeared {count} time(s)")

print("\nMost Frequently Occurring Number(s):", most_frequent, f"(Appeared {max_count} times)")
print("Unique Tuple:", unique_tuple)
```

```
Original List: [12, 5, 8, 12, 3, 5, 9, 8, 15, 3, 20, 12, 9, 25, 15]
Unique List: [12, 5, 8, 3, 9, 15, 20, 25]

Frequency Report:
Number 12: appeared 3 time(s)
Number 5: appeared 2 time(s)
Number 8: appeared 2 time(s)
Number 3: appeared 2 time(s)
Number 9: appeared 2 time(s)
Number 15: appeared 2 time(s)
Number 20: appeared 1 time(s)
Number 25: appeared 1 time(s)

Most Frequently Occurring Number(s): [12] (Appeared 3 times)
Unique Tuple: (12, 5, 8, 3, 9, 15, 20, 25)
```

Problem 3: Tuple-Based Inventory Management Difficult

A computer shop stores its inventory using tuples.

```python
inventory = [
    ("KB101", "Keyboard", 25, 450.00),
    ("MS102", "Mouse", 40, 250.00),
    ("HD103", "Hard Drive", 12, 2800.00),
    ("MN104", "Monitor", 8, 6500.00),
    ("USB105", "USB Drive", 50, 350.00)
]
```

Each tuple contains `(product_code, product_name, quantity, unit_price)`.

Requirements:

1. Calculate the total value of each product using quantity multiplied by unit price.
2. Calculate the total inventory value.
3. Identify products with fewer than 15 units.
4. Find the product with the highest total inventory value.
5. Sort the inventory by unit price in descending order.
6. Generate a new list of tuples containing product name, quantity, and total value.

Challenge: Allow the user to enter a product code and display the matching product information. Display an appropriate message if the code does not exist.

```python
inventory = [
    ("KB101", "Keyboard", 25, 450.00),
    ("MS102", "Mouse", 40, 250.00),
    ("HD103", "Hard Drive", 12, 2800.00),
    ("MN104", "Monitor", 8, 6500.00),
    ("USB105", "USB Drive", 50, 350.00)
]

summary = [(name, qty, qty * price) for code, name, qty, price in inventory]

print("Total Value per Product:")

for name, qty, value in summary:
    print(f"{name}: {value:.2f}")


total = sum(value for name, qty, value in summary)

print(f"\nTotal Inventory Value: {total:.2f}")

low_stock = [name for code, name, qty, price in inventory if qty < 15]

print("Products with fewer than 15 units:", low_stock)

highest = max(summary, key=lambda x: x[2])

print("Product with highest total inventory value:", highest[0])

sorted_inventory = sorted(inventory, key=lambda x: x[3], reverse=True)

print("\nInventory sorted by unit price (descending):")

print(sorted_inventory)

print("\nNew list of tuples (Product Name, Quantity, Total Value):")

print(summary)

search code = input("\nEnter product code to search: ").upper()
```

```python
search_code = input("\nEnter product code to search: ").upper()

found = False

for code, name, qty, price in inventory:
    if code.upper() == search_code:
        print(f"Product Found: Code={code}, Name={name}, Quantity={qty}, Unit Price={price:.2f}")
        found = True
        break

if not found:
    print("Product code does not exist.")
```

```
Total Value per Product:
Keyboard: 11250.00
Mouse: 10000.00
Hard Drive: 33600.00
Monitor: 52000.00
USB Drive: 17500.00

Total Inventory Value: 124350.00
Products with fewer than 15 units: ['Hard Drive', 'Monitor']
Product with highest total inventory value: Monitor

Inventory sorted by unit price (descending):
[('MN104', 'Monitor', 8, 6500.0), ('HD103', 'Hard Drive', 12, 2800.0), ('KB101', 'Keyboard', 25, 450.0), ('USB105', 'USB Drive', 50, 350.0), ('MS102', 'Mouse', 40, 250.0)]

New list of tuples (Product Name, Quantity, Total Value):
[('Keyboard', 25, 11250.0), ('Mouse', 40, 10000.0), ('Hard Drive', 12, 33600.0), ('Monitor', 8, 52000.0), ('USB Drive', 50, 17500.0)]

Enter product code to search: Mouse
Product code does not exist.
```

Problem 4: Tuple-Based Employee Payroll Analysis Difficult

Develop a payroll analysis program using the following data.

Python

```python
employees = [
    ("E001", "Alice", 40, 250),
    ("E002", "Brian", 45, 300),
    ("E003", "Clara", 38, 280),
    ("E004", "David", 50, 350),
    ("E005", "Ella", 42, 320)
]
```

Each tuple contains `(employee_id, name, hours_worked, hourly_rate)`.

Assume overtime is paid at 1.5 times the hourly rate for hours exceeding 40.

Requirements:

1. Calculate regular pay and overtime pay for each employee.
2. Calculate the gross salary of each employee.
3. Display employees who worked overtime.
4. Identify the employee with the highest gross salary.
5. Calculate the average gross salary.
6. Store the results as tuples in a new list.
7. Sort employees by gross salary from highest to lowest.

Challenge: Allow the user to search for an employee by ID and display the employee's payroll details.

```python
employees = [
    ("E001", "Alice", 40, 250),
    ("E002", "Brian", 45, 300),
    ("E003", "Clara", 38, 280),
    ("E004", "David", 50, 350),
    ("E005", "Ella", 42, 320)
]

payroll = []
overtime = []

for emp_id, name, hours, rate in employees:
    reg_pay = min(hours, 40) * rate
    ot_pay = max(hours - 40, 0) * rate * 1.5
    gross = reg_pay + ot_pay
    payroll.append((emp_id, name, reg_pay, ot_pay, gross))

    if hours > 40:
        overtime.append(name)

print("EMPLOYEE PAYROLL SUMMARY")
for emp_id, name, reg_pay, ot_pay, gross in payroll:
    print(f"ID: {emp_id} | Name: {name} | Regular Pay: {reg_pay:.2f} | Overtime Pay: {ot_pay:.2f} | Gross Salary: {gross:.2f}")

highest = max(payroll, key=lambda x: x[4])
print(f"\nEmployees who worked overtime: {overtime}")
print(f"Employee with highest gross salary: {highest[1]} ({highest[4]:.2f})")

average = sum(emp[4] for emp in payroll) / len(payroll)
print(f"Average Gross Salary: {average:.2f}")

sorted_payroll = sorted(payroll, key=lambda x: x[4], reverse=True)
print("\nEmployees Sorted by Gross Salary (Highest to Lowest):")
print(sorted_payroll)

search_id = input("\nEnter Employee ID to search: ").upper()
found = False

for emp_id, name, reg_pay, ot_pay, gross in payroll:
    if emp_id == search_id:
        print(f"\nDetails for {emp_id}:")
        print(f"Name: {name}")
        print(f"Regular Pay: {reg_pay:.2f}")
        print(f"Overtime Pay: {ot_pay:.2f}")
        print(f"Gross Salary: {gross:.2f}")
        found = True
        break

if not found:
    print("Employee ID not found.")
```

```
EMPLOYEE PAYROLL SUMMARY
ID: E001 | Name: Alice | Regular Pay: 10000.00 | Overtime Pay: 0.00 | Gross Salary: 10000.00
ID: E002 | Name: Brian | Regular Pay: 12000.00 | Overtime Pay: 2250.00 | Gross Salary: 14250.00
ID: E003 | Name: Clara | Regular Pay: 10640.00 | Overtime Pay: 0.00 | Gross Salary: 10640.00
ID: E004 | Name: David | Regular Pay: 14000.00 | Overtime Pay: 5250.00 | Gross Salary: 19250.00
ID: E005 | Name: Ella | Regular Pay: 12800.00 | Overtime Pay: 960.00 | Gross Salary: 13760.00

Employees who worked overtime: ['Brian', 'David', 'Ella']
Employee with highest gross salary: David (19250.00)
Average Gross Salary: 13580.00

Employees Sorted by Gross Salary (Highest to Lowest):
[('E004', 'David', 14000, 5250.0, 19250.0), ('E002', 'Brian', 12000, 2250.0, 14250.0), ('E005', 'Ella', 12800, 960.0, 13760.0), ('E003', 'Clara', 10640, 0.0, 10640.0), ('E001', 'Alice', 10000, 0.0, 10000.0)]

Enter Employee ID to search: E001

Details for E001:
Name: Alice
Regular Pay: 10000.00
Overtime Pay: 0.00
Gross Salary: 10000.00
```

Problem 5: Leaderboard Ranking System

Difficult

Develop a leaderboard for a programming competition.

Python

Run

```python
contestants = [
    ("C001", "Alex", (85, 90, 88)),
    ("C002", "Bea", (95, 80, 92)),
    ("C003", "Chris", (90, 90, 90)),
    ("C004", "Dana", (95, 85, 87)),
    ("C005", "Evan", (88, 92, 91)),
    ("C006", "Gia", (95, 85, 87))
]
```

Each contestant has three problem scores.

Requirements:

1. Calculate the total and average score of each contestant.
2. Rank contestants by total score from highest to lowest.
3. If two contestants have the same total score, rank them by their highest individual problem score.
4. If both criteria are equal, preserve their original order.
5. Store the final leaderboard as a list of tuples.
6. Display the top three contestants.
7. Identify contestants whose average score is at least 90.

Challenge: Implement the ranking logic using a custom sorting function or sorting key. Do not modify the

```python
contestants = [
    ("C001", "Alex", (85, 90, 88)),
    ("C002", "Bea", (95, 80, 92)),
    ("C003", "Chris", (90, 90, 90)),
    ("C004", "Dana", (95, 85, 87)),
    ("C005", "Evan", (88, 92, 91)),
    ("C006", "Gia", (95, 85, 87))
]

leaderboard = []
qualified = []

for c_id, name, scores in contestants:
    total = sum(scores)
    avg = total / len(scores)
    highest = max(scores)
    leaderboard.append((c_id, name, total, avg, highest))

    if avg >= 90:
        qualified.append(name)

leaderboard.sort(key=lambda x: (x[2], x[4]), reverse=True)

final_leaderboard = [
    (c_id, name, total, round(avg, 2))
    for c_id, name, total, avg, highest in leaderboard
]

print("LEADERBOARD:")
for rank, (c_id, name, total, avg) in enumerate(final_leaderboard, 1):
    print(f"Rank {rank}: ID: {c_id} | Name: {name} | Total Score: {total} | Average Score: {avg:.2f}")

print("\nTOP 3 CONTESTANTS:")
for rank, (c_id, name, total, avg) in enumerate(final_leaderboard[:3], 1):
    print(f"Rank {rank}: {name} (Total: {total}, Average: {avg:.2f})")

print("\nContestants with average score of at least 90:")
print(qualified)

print("\nFinal Leaderboard (List of Tuples):")
print(final leaderboard)
```

```
LEADERBOARD:
Rank 1: ID: C005 | Name: Evan | Total Score: 271 | Average Score: 90.33
Rank 2: ID: C003 | Name: Chris | Total Score: 270 | Average Score: 90.00
Rank 3: ID: C002 | Name: Bea | Total Score: 267 | Average Score: 89.00
Rank 4: ID: C004 | Name: Dana | Total Score: 267 | Average Score: 89.00
Rank 5: ID: C006 | Name: Gia | Total Score: 267 | Average Score: 89.00
Rank 6: ID: C001 | Name: Alex | Total Score: 263 | Average Score: 87.67

TOP 3 CONTESTANTS:
Rank 1: Evan (Total: 271, Average: 90.33)
Rank 2: Chris (Total: 270, Average: 90.00)
Rank 3: Bea (Total: 267, Average: 89.00)

Contestants with average score of at least 90:
['Chris', 'Evan']

Final Leaderboard (List of Tuples):
[('C005', 'Evan', 271, 90.33), ('C003', 'Chris', 270, 90.0), ('C002', 'Bea', 267, 89.0), ('C004', 'Dana', 267, 89.0), ('C006', 'Gia', 267, 89.0), ('C001', 'Alex', 263, 87.67)]
```

Problem 6: Advanced List and Tuple Data Analyzer Difficult

Create a menu-driven program that analyzes a list of numerical data.

Sample data:

```python
data = [
    14, 22, 18, 14, 35, 27, 22, 40,
    18, 14, 31, 27, 45, 22, 35, 50,
    18, 40, 27, 31
]
```

Your program must provide the following menu:

```
===== DATA ANALYZER =====
1. Display Original Data
2. Display Sorted Data
3. Remove Duplicates
4. Display Frequency Table
5. Display Minimum and Maximum
6. Display Mean and Median
7. Display Second-Largest Distinct Value
8. Convert Data into a Tuple
9. Search for a Value
10. Exit
```

Requirements:

1. Use functions to implement each menu operation.
2. Use a list to store the original data and a tuple for the converted data.
3. Remove duplicates without using `set()`.
4. Calculate the mean and median without relying on statistics libraries.
5. Find the second-largest distinct value.
6. Display all values with the highest frequency.
7. Continue displaying the menu until the user selects Exit.
8. Validate user input to prevent program crashes from invalid menu choices or nonnumeric search values.

Challenge: Design your functions so that the original dataset remains unchanged when sorting or removing duplicates.

```python
data = [
    14, 22, 18, 14, 35, 27, 22, 40,
    18, 14, 31, 27, 45, 22, 35, 50,
    18, 40, 27, 31
]

def sort_data(data):
    return sorted(data)

def remove_duplicates(data):
    return list(dict.fromkeys(data))

def frequency_table(data):
    counts = {num: data.count(num) for num in remove_duplicates(data)}
    print("\nFrequency Table:")
    for num, count in counts.items():
        print(f"Value {num}: {count} time(s)")

    highest = max(counts.values())
    most_frequent = [num for num, count in counts.items() if count == highest]
    print(f"Highest Frequency Value(s) (appeared {highest} times):", most_frequent)

def display_min_max(data):
    print("\nMinimum Value:", min(data))
    print("Maximum Value:", max(data))

def display_mean_median(data):
    sorted_data = sort_data(data)
    n = len(data)
    mean = sum(data) / n

    if n % 2 == 1:
        median = sorted_data[n // 2]
    else:
        median = (sorted_data[n // 2 - 1] + sorted_data[n // 2]) / 2

    print(f"\nMean: {mean:.2f}")
    print("Median:", median)

def display_second_largest(data):
    unique = sort_data(remove_duplicates(data))
```

```
def display_second_largest(data):
    unique = sort_data(remove_duplicates(data))
    if len(unique) >= 2:
        print("\nSecond-Largest Distinct Value:", unique[-2])
    else:
        print("\nNot enough distinct values to find second-largest.")

def search_value(data):
    user_input = input("\nEnter a number to search: ")

    if user_input.lstrip('-').isdigit():
        target = int(user_input)
        count = data.count(target)

        if count:
            print(f"Value {target} was found in the dataset {count} time(s).")
        else:
            print(f"Value {target} is not in the dataset.")
    else:
        print("Invalid input. Please enter a valid numeric value.")

while True:
    print("\n----- DATA ANALYZER -----")
    print("1. Display Original Data")
    print("2. Display Sorted Data")
    print("3. Remove Duplicates")
    print("4. Display Frequency Table")
    print("5. Display Minimum and Maximum")
    print("6. Display Mean and Median")
    print("7. Display Second-Largest Distinct Value")
    print("8. Convert Data into a Tuple")
    print("9. Search for a Value")
    print("10. Exit")

    choice = input("\nEnter your choice (1-10): ")

    if choice == '1':
        print("\nOriginal Data:")
        print(data)
```

```
        elif choice == '2':
            print("\nSorted Data:")
            print(sort_data(data))
        elif choice == '3':
            print("\nData without Duplicates:")
            print(remove_duplicates(data))
        elif choice == '4':
            frequency_table(data)
        elif choice == '5':
            display_min_max(data)
        elif choice == '6':
            display_mean_median(data)
        elif choice == '7':
            display_second_largest(data)
        elif choice == '8':
            print("\nConverted Data into Tuple:")
            print(tuple(data))
        elif choice == '9':
            search_value(data)
        elif choice == '10':
            print("\nExiting program...")
            break
        else:
            print("\nInvalid choice. Please enter a number from 1 to 10.")
```

```
...
    ----- DATA ANALYZER -----
    1. Display Original Data
    2. Display Sorted Data
    3. Remove Duplicates
    4. Display Frequency Table
    5. Display Minimum and Maximum
    6. Display Mean and Median
    7. Display Second-Largest Distinct Value
    8. Convert Data into a Tuple
    9. Search for a Value
    10. Exit

    Enter your choice (1-10): 9
```